DEV400

Schema Design

MongoDB Developer Extension

Release: 20260220

Topics we cover

- BSON
- Container Types - Arrays vs Documents
- Document Design Fundamentals
- Linking Models
- Payload and Process Fields
- Dynamic Schemas
- Schema Design Patterns

BSON

Binary Serialized Object Notation (http://bsonspec.org/spec.html)

- Language-independent binary data specification
- Used internally and for network traffic
- Serialized and Deserialized by MongoDB Drivers
- Contains a list of named, typed values:

  Type, Fieldname, <length>, Data

  Type, Fieldname, <length>, Data

How do we find a value in a BSON document?

How long does that take relative to the size of it?

Why have Objects not just fields?

- BSON is a format in which zero or more ordered key/value pairs are stored as a single entity
- We call this entity a document
- As field names are stored inline - large field names take more storage and cache and are therefore an anti-pattern
- Answers at the end

BSON Grammar

From the BSON Specification:

```
document    ::= int32 e_list unsigned_byte(0)
e_list      ::= element e_list
            |   ""

element ::= signed_byte(1) e_name double    64-bit FP
        |   signed_byte(2) e_name string    UTF-8 string
        |   signed_byte(3) e_name document  Embedded document
        |   signed_byte(4) e_name document  Array

        ...

        |   signed_byte(19) e_name decimal128 128-bit FP
```


The specification defines a BSON document to contain the size of the document (int32), followed by a list of elements, terminated by a zero byte.

Elements have a type, followed by the name of the element, followed by the value of the element. An element with type '1' contains a double-precision floating-point value. Strings are null-terminated.

The slide shows the definitions of an embedded document and an array differ only in the type value.

https://bsonspec.org/spec.html

Container Types

Container types (Document and Array) define the Document Model

Advantages:

- Fewer Joins
- Fewer Indexes
- Faster Updates
- Faster Retrievals
- Simple Scalability

Document Model Design allows you to define your schema in terms of what your application needs to perform best. Data for a single object can be co-located in a single document. No more need for ORMs!

Arrays vs Documents

Arrays and Documents differ by one byte – storage is the same

There are no Arrays, just Objects

Objects are just lists of fields, types, and values

```
Document:   {list: {0 : "Red", 1 : "Orange", 2 : "Yellow"}}
Array:      {list: [ "Red", "Orange", "Yellow" ]}
```

Both will have identical BSON encodings:

```
list.0 "Red" … list.1 "Orange" … list.2 "Yellow"
```


Arrays are stored as BSON documents with integer values for the keys, starting with 0 and continuing sequentially.

Arrays vs Documents

The difference is in how they map to a programming language and what constraints

- Can $push to an array, but not to a document
- Arrays can contain duplicates
- Arrays cannot be sparse, must store nulls for empty values

Large Arrays or large, flat Documents are equally bad

- Large or unbounded arrays can have a significant speed impact

Arrays are often used where documents would be a better choice!

- As an array continues to grow, reading and building indexes on that array gradually impact performance. A large, growing array can strain application resources and put your documents at risk of exceeding the BSON Document Size limit.
- Often it is better to use a document instead of an array.

Array Schema vs Document Schema

Redesign to be smaller

In this case, 30% smaller documents

Reduces memory/cache usage

- Disk usage relatively unaffected
- Documents in disk are usually compressed

```
> var doc = {
      results: [
                  {player: 'john', score: 25},
                  {player: 'fred', score: 20},
                  {player: 'sarah', score : 50}   ]
      }

> bsonsize(doc)
128

> var doc = {
      results: {
  john : {score: 25},
  fred : {score: 20},
  sarah: {score: 50}
            }
      }

> bsonsize(doc)
86
```


- Could we make that smaller still?
- Notably, the first is a fixed schema; the second is dynamic/schemaless
- The second would require a wildcard index if we wanted to be able to search it - and that would be larger as it contains the keys as well as values.
- The dynamic schema assumes we always know the name of the player whose score we want to fetch

When to use Arrays

Use Arrays when

- Just need to $push to the end
- Unique identifiers not required
- Sparse data is minimal
- Only values need to be indexed

Otherwise, consider Documents

- Projection is faster than $filter
- Querying a single member is faster

One big advantage of Arrays over Joined tables is not needing an indexed (foreign) key for each element

One advantage of using an Array to store a number of sub elements - versus putting them in their own collection and joining them is at the very least you don't need the _id value and index on _id you would have in the other collection.

Storing small things like sensor readings or bank transactions (one per document in MongoDB) would likely be very inefficient, doubly so as _id is technically a secondary index and we not only have to store a value for _id (technically we could be clever about that and use a value we were storing anyway) but we need to add an index too. Arrays let us store multiple values without needing an _id value for each element or having to store keys to link records.

If there are times we don't get sensor readings or bank transactions, the entire data can still be stored in the array with NULL values for every reading that we don't get (sparse data).

Antipattern: Large/Unbounded Arrays

Use case: Record number of events per minute of a 24-hour day. Increment the value of a specific minute by one when an event occurs in that minute.

Solution 1: Use a single document with a 2-D array ([day][num-events-per-minute]) -> [number-of-days]*1440 elements

Solution 2: Use a single document for each day with a 1-D array ([num-events-per-minute]) -> each document with 1440 elements

Which will be more performant?

Arrays are useful because you can just append elements to them, but they are dangerous for the same reason. An unbounded array can grow only up until the maximum document size of 16MB, but long before then performance will become unacceptable.

The reason for this is that even if you are interested in only a single array element, MongoDB must read the entire document into memory to find that element. Updates require the entire document to be written to disk. (Partial reads or writes would make retaining ACID-compliance extremely difficult.)

To demonstrate this, say we want to store just the number of events which happen during a particular minute of a particular day. Because we aren't interested in additional details (like sensor readings, for example), we can simply use an array with each element corresponding to a particular minute.

Because we want to store this data for multiple days, our first decision is to create a single document with a 2-D array. If an event occurs during the 3rd minute of the 15th day, we would increment the element [14][2]. For each day we would push 1440 empty elements on to the array. This means our document will grow by around 12.9kb per day.

Alternatively, we could use a new document for each day, storing the number of the day in the _id field (which is automatically indexed!) and a single 1-D array with 1440 elements. To record the above event, we would increment the element 14 in the document with _id:2. Each document would be a full 13kb in size, so we have a slight bit of overhead.

Assuming we can use a bulk operation to update a full day's worth of events at a time, which method will be more performant? How will each method scale?

Large/Unbounded Array

For each day, we push a ‘column’ of 1440 elements to the array, and update it atomically.

The Document size grows each day

Write times trend upwards

```
totalstart = new Date()
myarray = []
myarray = new Array(1440).fill(0)
db.events.deleteOne({_id:"test"})
for(day=0;day<50;day++) {
    print("Day: "+day)
    db.events.updateOne({_id:"test"},{$push: {a: myarray} }, {upsert:true})
    eventtimes = []
    for(x=0;x<2000;x++)  { eventtimes.push(Math.floor(Math.random()*1440)) }
    bulk = db.events.initializeUnorderedBulkOp()
    for(x=0;x<2000;x++) {
        inc = {}
        d = eventtimes[x]
        inc[`a.${day}.${d}`] = 1
        bulk.find({_id:"test"}).updateOne({$inc:inc})
    }
    start = new Date()
    bulk.execute()
    time = new Date() - start
    docsize = bsonsize(db.events.findOne({_id:"test"}))
    print("Update time: " + time + "ms  / Document size: " + (docsize/1024).toFixe
d(1) + "KB")
}
totaltime = new Date() - totalstart
print("Total time: " + totaltime + "ms")
```


This is best demonstrated using a local mongod instance or a dedicated Atlas cluster.


```
totalstart = new Date()
myarray = []
myarray = new Array(1440).fill(0)
db.events.deleteOne({_id:"test"})
for(day=0;day<50;day++) {
    print("Day: "+day)
    db.events.updateOne({_id:"test"},{$push: {a: myarray} }, {upsert:true})
    eventtimes = []
    for(x=0;x<2000;x++)  { eventtimes.push(Math.floor(Math.random()*1440)) }
    bulk = db.events.initializeUnorderedBulkOp()
    for(x=0;x<2000;x++) {
        inc = {}
        d = eventtimes[x]
        inc[`a.${day}.${d}`] = 1
        bulk.find({_id:"test"}).updateOne({$inc:inc})
    }
    start = new Date()
    bulk.execute()
    time = new Date() - start
    docsize = bsonsize(db.events.findOne({_id:"test"}))
    print("Update time: " + time + "ms  / Document size: " + (docsize/1024).toFixed(1)
+ "KB")
}
totaltime = new Date() - totalstart
print("Total time: " + totaltime + "ms")
```

Bounded Array

For each day, we create a new document

Only have to deal with a 1-D array

Document sizes constant

Write times stable

```
totalstart = new Date()
myarray = []
myarray = new Array(1440).fill(0)
for(day=0;day<50;day++) {
    db.events.replaceOne({_id:day},{a:myarray},{upsert:true})
    print("Day "+day)
    eventtimes = []
    for(x=0;x<2000;x++)  { eventtimes.push(Math.floor(Math.random()*1440)) }
    bulk = db.events.initializeUnorderedBulkOp()
    for(x=0;x<2000;x++) {
        inc = {}
        d = eventtimes[x]
        inc[`a.${d}`] = 1
        bulk.find({_id:day}).updateOne({$inc:inc})
    }
    start = new Date()
    bulk.execute()
    time = new Date() - start
    docsize = bsonsize(db.events.findOne({_id:day}))
    print("Update time: " + time + "s  / Document size: " + (docsize/1024).toFixed
(1) + "KB")
}
totaltime = new Date() - totalstart
print("Total time: " + totaltime + "ms")
```


This is best demonstrated using a local mongod instance or a dedicated Atlas cluster.


```
totalstart = new Date()
myarray = []
myarray = new Array(1440).fill(0)
for(day=0;day<50;day++) {
    db.events.replaceOne({_id:day},{a:myarray},{upsert:true})
    print("Day "+day)
    eventtimes = []
    for(x=0;x<2000;x++)  { eventtimes.push(Math.floor(Math.random()*1440)) }
    bulk = db.events.initializeUnorderedBulkOp()
    for(x=0;x<2000;x++) {
        inc = {}
        d = eventtimes[x]
        inc[`a.${d}`] = 1
        bulk.find({_id:day}).updateOne({$inc:inc})
    }
    start = new Date()
    bulk.execute()
    time = new Date() - start
    docsize = bsonsize(db.events.findOne({_id:day}))
    print("Update time: " + time + "s  / Document size: " + (docsize/1024).toFixed(1)
+ "KB")
}
totaltime = new Date() - totalstart
print("Total time: " + totaltime + "ms")
```

Document Design Fundamentals

Document Model vs Relational Model

RDBMS – One correct design

- Data Normalised to 3NF
- Design based on the data, not the usage
- Reality is more nuanced

Document – Design Patterns

- Many design options
- Designed for usage
- Retrieval (Fast retrieval of required info)
- Updates (Atomic update of business logic)

Document design targets fast data retrieval and updates rather than normalization

Schema Design in MongoDB

Schema is defined in the Application

Design is part of each phase in its lifetime

Schema should focus on the needs of the Application:

- What data does the App need?
- How fast does it need it?
- Is data duplication a reasonable tradeoff for performance?

Schema Validation

MongoDB has a flexible schema by default

Schema Validation allows for a fixed schema

- Fixed Schemas can define:
  - Required fields
  - Required data types
  - Required field values (eg. number ranges or string enumerations)

MongoDB has a flexible schema model (documents in a collection do not need to have the same fields or data types by default)
But once you've established an application schema, you can use schema validation to ensure there are no unintended schema changes or improper data types.

Schema Validation Use Cases

Flexible Schema is good for new applications:

- Functional requirements not yet fully defined
- Fields may change over time
- New functionality may require additional fields

Fixed Schema is good for established applications

- Fields and data types are set and unlikely to change
- Functionality depends on specific fields
- Data entry errors need to be reported

How do we deal with existing data when we enable Schema Validation?

Another use case for Schema Validation is if we do not have complete control over the applications which are writing to the database. We can use Schema Validation to ensure that only applications adhering to our schema can write to our database.

Usually we will only enable Schema Validation once our application is relatively mature, but by this point the database may already contain large amounts of data that does not adhere to the schema.

Enabling Schema Validation has no effect on data already in the database!

Schema Validation Levels

Strict Schema Validation (default level):

- All inserted documents must be valid
- All updated documents must result in a valid document

Moderate Schema Validation:

- All inserted documents must be valid
- All updated documents, *which were already valid*, must result in a valid document
- Updates to existing, invalid documents, do not have to result in a valid document

MongoDB throws an error if validation fails.

The default schema validation level is 'strict'. This means that all write operations (inserts, updates) must result in a document which is "valid", ie. it adheres to the schema. If not, MongoDB will reject the write operation and throw an error.

We can specify 'moderate' validation if we have existing documents which do not adhere to the schema, and we can't or don't want to modify them to match the schema, but we still need to perform updates.
With moderate validation, if a document is not valid, we can update it such that the updated version is *also* not valid. However, once we update a document such that it adheres to the schema, all subsequent updates must adhere to the schema as well.

By default, MongoDB will throw an error if validation fails and reject the write operation. However, we can set the validationAction in our validator to "warn". This will log the violation, but allow the operation to proceed. This is most useful when bulk inserting data created before the schema was established.

Schema Validation Example

Schema Validation lets you define:

- Field types
- Field value ranges / enums
- Required fields
- Dynamic validation rules

To add schema validation to an existing collection, use:

```
db.runCommand(
      {
      "collMod": "MyCollection",
      "validator": {$jsonSchema:{ ... }},
      "validationLevel": "moderate"
      }
)
```

```
db.createCollection("books", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: [ "title", "year" ],
         properties: {
            title: {
               bsonType: "string",
               description: "'title' must be a string and is required"
            },
            year: {
               bsonType: "int",
               maximum: 3000,
               description: "'year' must exist and be an integer no more
than 3000"
            },
            price: {
               bsonType: [ "decimal128" ],
               description: "'price' must be a decimal128 if the field
exists"
            }
         }
      }
```


Schema Validation uses JSON Schema, with the added 'bsonType" keyword for defining field types.

You can define required fields, required data types, ranges and enums for field values, and dynamic validation rules. For example you can define a field that depends on the value of another field, and dynamically check that its value is as expected.

More details: https://www.mongodb.com/docs/manual/core/schema-validation/

Exercise - Schema Validation

In your cluster:

- Create a new Database called **`bookStore`**
- Create a new collection called **`books`** with the following restrictions:
  - Must include a unique ISBN (integer), the Year (integer), and rating (integer from 1 to 5)
  - Can include an author field, but this field must be a string
  - Can include a comments field, and this field must be an array.
- Test your schema validation by adding 2 new documents:

```
db.books.insertOne({ISBN:"101", year:2000, rating:5, title:"One", author:"TedBell"})
```

```
db.books.insertOne({ISBN:"102", rating:10, author:7, comments:["great", "funny"]})
```


Exercise Solution
In mongosh connect to your MDB cluster and run:
use bookStore

```
db.createCollection( "books",
{ "validator": {
    $jsonSchema: {
      bsonType: "object",
      required: [ "ISBN", "year", "rating" ],
      properties: {
        ISBN: {bsonType: "int"},
        year: { bsonType: "int"},
        rating: { bsonType: "int",minimum: 1,maximum: 5},
        author: { bsonType: "string"},
        comments: {bsonType:"array"}
      }
    }
  }
})
```

Note: To implement a uniqueness of a field value across documents, we need to use unique indexes.
db.books.createIndex({ "ISBN": 1 }, { unique: true })

Schema Design Considerations

1. The data the application needs
2. Application's read usage of the data
3. Application's write usage of the data

Document Design Basics

To Link or Embed?

- How often do I want the Embedded info?
- How often does the Embedded info change?
- Do I need to search with the Embedded info?
- Do I need the latest version or the same version?
- Is the Embedded info private/sensitive?

What about an Invoice Address, link, or embed?

Linking or Embedding data are two approaches to consider.

If you are embedding data, note that it will be visible to anyone with access to the collection. Set up your security accordingly if you are embedding sensitive information.

Linking models

Link types

One-to-One Relationships

One-to-Many Relationships

Many-to-Many Relationships

Link types will be familiar from the relational world

One-to-One Linked

```
//Either side can track relationship
book = {
    _id: "B456",
    title: "The Great Gatsby",
    slug: "9781857150193-the-great-gatsby",
    author: "A123",
    // Other fields follow…
}
author = {
    _id: "A123",
    firstName: "F. Scott",
    lastName: "Fitzgerald",
    book: "B456" //Alternative to store in author
}
authorofbook = db.authors.find({_id : book.author})
bookbyauthor = db.books.find({author : author._id})
```


If you store at both ends you don't need an additional index.

Note: We are assuming in this particular slide that there is just one book stored per author.

One-to-One Embedded

```
book = {
    _id: "B456",
    title: "The Great Gatsby",
    slug: "9781857150193-the-great-gatsby",
    author: {
        firstName: "F. Scott",
        lastName: "Fitzgerald"
    }
    // Other fields follow…
}
```


- Fast to fetch, fast and efficient for query or aggregation
- Less efficient even 1:1 if, say, author info is large and seldom used as the whole document pulled into the cache - not just some fields.
- See later design patterns.

One-to-Many: Array in Parent

```
author = {
    _id: "A123",
    firstName: "F. Scott",
    lastName: "Fitzgerald",
    books: ["B456", "B789", "B20"]
}

booksbyauthor = db.books.find({_id: { $in : author.books}})
authorofbook = db.author.find({books : book._id})
```


This is where arrays become a really powerful tool for data modeling.

One-to-Many: Scalar in Child

```
book1 ={
    _id: "B456",
    title: "The Great Gatsby",
    slug: "9781857150193-the-great-gatsby",
    author: "A123",
    // Other fields follow…
}
book2 = {
    _id: "B789",
    title: "This Side of Paradise",
    slug: "9780679447238-this-side-of-paradise",
    author: "A123",
// Other fields follow…
}
authorofbook = db.authors.find({_id:book.author})
booksbyauthor = db.books.find({author:author._id})
```

Many-to-Many: Array in either side

```
book = {
    _id: "B456",
    title: "The Great Gatsby",
    authors: ["A123", "A5"]
    // Other fields follow…
}


authorsofbook = db.authors.find({_id:{$in:book.authors}})
booksbyauthor = db.books.find({authors: author._id}) //Query against array

author = {
    _id: "A123",
    firstName: "F. Scott",
    lastName: "Fitzgerald",
    books: ["B456"] //Alternative - don't need both ends
}
```


Arrays can also model many-to-many relationships very easily

Exercise - Blog Post Scenario

A developer is designing a blogging platform where users can create posts, comment on them, and like them. The application needs to display each post along with its comments and likes on the same page. What approach should be taken when designing your MongoDB schema to efficiently handle the retrieval of posts with their comments and likes? Assume the application doesn't have many users.

A) Create a separate collection for posts, comments, and likes, and use $lookup during query.

B) Embed comments and likes within the post documents as an array of subdocuments.

C) Store all data in a single large document per user to minimize collections.

D) Use a relational database model to handle the relationships.

Payload fields vs Process fields

Two Types of Fields

Payload – Data we just store and retrieve

Processing – metadata we examine inside MongoDB

- Querying
- Aggregating
- Filtering and Projecting
- Using for workflow
- Using to implement patterns

Payload might be predefined, sometimes better to ignore that and add Processing fields
For example, existing corporate data/object models, might be used only for storage
JSON files you want to store is Payload, fields you want to filter by are processing fields

- There are two types of terminology we classify fields into:
  - Payload - data we just store and retrieve from the app but the database server never looks at these fields or do anything with them
  - Processing - metadata we examine in MDB- fields the server uses them to query/look at the values
- Corporate data models can be stored in MDB but sometimes traleting to JSON is not necessary if the fields are not processing fields - see example (next slide)

Example

Need to store XML objects defined as part of the business in a MongoDB collection

First idea: convert XML to JSON and keep it in MongoDB

- But only ever query a few fields
- Normally always want just the latest
- XML structures are complex or dynamic
- Need to write a reverse conversion too

Conversion XML <-> JSON is not a good idea, it's expensive and difficult

Best option:

- Extract the processing data (fields needed for queries like create date, etc.)
- Store processing fields in documents
- Put the XML itself in a Binary (compressed) – it's a payload field

Example - Storing XML compressed as binary in MongoDB is usually better than converting it to JSON.

Dynamic Schema

Three Dynamic Models

Evolutionary Dynamic Schema

Payload-Driven Dynamic Schema

Data Driven Dynamic Schema

Evolutionary Dynamic Schema

Schema changes as application changes - big benefit to using MongoDB

Slow changes to the application, but no big conversion needed

- Include a schema version number with the data as a field
- Retain the ability to read previous schemas in application
- Either convert in the background or on modification
- Loosely coupled Database objects and Code Objects

Schema versioning can be used to achieve a dynamic schema that changes with your application.

Payload-Driven Dynamic Schema

Developer has **no control** over what data goes in – the application's purpose is to store some arbitrary data
Schema is unpredictable

- Cannot optimize much for arbitrary data
- Performance will be poor
- Wildcard indexes can be used but are not a great solution

Think Payload vs Processing:
Can you optimize by using metadata?

Data Driven Dynamic Schema

Field Names are the data values

- Uncomfortable new concept for many designers
- Requires a truly dynamic coding approach
- Clean and performant for many things
- Can be used to do some amazing optimization of retrieval using Tries

Very simple example:

```
results: {   john: {score: 25},
             fred: {score: 20},
             sarah: {score: 50} }
```


- Imagine a document where you need many thousands of keys - you can have each character be a hierarchy level like {t : {h: { i : { s: 5}}}}
- This can make for optimal internal traversal and retrieval.
- Tries are a data structure based on prefix trees. https://en.wikipedia.org/wiki/Trie

Design Patterns

What are Design Patterns?

Reusable solutions to common problems

In Schema Design:

- Provide generalised solutions to common design problems in the form of templates
- Powerful tool for Developers/Data Modellers
  - Important to apply patterns appropriately

Common Design Patterns

Inheritance Pattern

Attribute Pattern

Bucket Pattern

Computed Pattern

Versioning Pattern

Subset Pattern

Outlier Pattern

Extended Reference Pattern

Design patterns provide generalised solutions to common design problems in the form of templates.
They are a powerful tool for software developers. However, it is important to apply patterns appropriately.
Using the incorrect pattern for a situation or applying a design pattern to a trivial solution can overcomplicate the code and lead to maintainability issues.

Inheritance Pattern

Used when Documents are *mostly* similar and should be queried together

Use Cases:

- Single Views
- Product Catalog
- Content Management

The Inheritance Pattern uses MongoDB's dynamic schema to allow for "polymorphism" - storing different objects together by grouping them by their common attributes.

A Single View is an unified representation of relevant or related information about an entity. For example, in a collection where we are storing all types of insurance policies (house, car, pet), we could use an aggregation to find all insurance policies opened in the past year in a specific area and compute its overall risk assessment.

In a Product Catalog, every product has a name, type or category, and a price. Individual products will have other attributes depending on their type.

If we want to query across all types or categories, we can write queries for the common attributes and handle the individual types in our application.

Inheritance Pattern Example

Related, yet distinct entities stored in the same collection

Store distinctions in fields or subdocuments

Application code knows how to deal with each type

```
> db.vehicles.insertMany([
{ type: "car",
  make: "Ford",
  model: "Mustang",
  year: 2022,
  car: {
        seatingFor: 4 } },

{ type: "truck",
  make: "Ford",
  model: "F-150",
  year: 2021,
  truck: {
        maxPayload: 2440,
        maxTowing: 13500 } },

{ type: "motorcycle",
  make: "Ducati",
  model: "Panigale V4",
  year: 2023,
  hasSidecar: false }
]);

> db.vehicles.find({ make: "Ford" })
> db.vehicles.find({ year: {$gte: 2022} })
```


Note there are various ways of defining the “specifics” for each type of element.

You could simply use additional fields, as in the ‘motorcycle’ example, but this may not scale very well. An alternative is to have a subdocument with the same name as the type.

The Inheritance Pattern is useful when you often need to query over all your different types of documents. If these documents were in different collections, you would need multiple queries to get the same information.

If you need to index the distinct fields, you will need to consider the cases where not all documents contain a specific field.

Attribute Pattern

Used where documents have a number of searchable named attributes

- Not all values may be present in a given record
- Any new record might have a new field name not seen before

What systems might have that?
If each of the documents has different field names, what indexes would we need?
How does this differ from the Inheritance Pattern?

The Attribute pattern is common in e-commerce applications, where there may be hundreds of different product types, each one with its own distinct set of attributes.
In this case, creating a well-defined subdocument for each type does not scale.
For indexing different field names, we would need a wildcard index, but wildcard indexes can have high storage and maintenance costs.
Recall that in the Inheritance pattern, we had a small, finite number of subtypes, so it was feasible to write code to handle each one.

Attribute Pattern Example

Fixed schema

Array of attribute Objects

Index objects OR Members

Variations exist to optimize some cases:

- Simple Key-Value Object
- Array of Key-Value Pairs
- Flattened Hybrid

Common in e-commerce

```
> db.product.drop()

> record = {
  sku: "iphonexblack64",
  attrs: [
    { k: "colour", v: "black" },
    { k: "ram", v: 64 },
    { k: "manufacturer", v: "apple" },
    { k: "screensize", v: "5.5 inch" }
  ]
}

> db.product.insertOne(record)

> db.product.createIndex({"attrs":1}) //OR
> db.product.createIndex({"attrs.k":1, "attrs.v":1})
```


Instead of using different field names in each document, we create a mini key-value pair embedded schema.
We can then index on the field names and the values if needed.

If we have known keys, like “colour”, and we would like direct access to them, we could use a simple key-value object to improve read simplicity: { attrs: { colour: "red", size: "M" } }
Or, we could flatten out known, common attributes by storing them in the main document: {sku: “iphonexblack64”, colour: “black”, attrs: [{k: “ram”, v: 64}] }

Bucket Pattern

Most common design pattern - Strongly matches Document Model

Used to store many small, related data items

- Bank Transactions – related by account and date
- IoT Readings – related by sensor and date

Reduces index sizes up to 200X!
Speeds up retrieval of related data
Enables computed pattern
MongoDB 5.0 timeseries collections abstract this concept

- The Bucket Pattern is a bit like index organized storage in an RDBMS
- Store multiple closely related data points in one document - updating it to add them.

Bucket Pattern Example

Use update with a size limit

Upsert creates new buckets automatically

Can bucket on size, timestamp, or both

```
> reading = {sensor:5,value:22,time:new Date('2019-05-11')}

> query = {sensor:reading.sensor,count: {$lt:200} }

> updates = {
    $push: {readings: {ts: reading.time, v: reading.value }},
    $inc: { count: 1 }
}

> db.iot.updateOne(query,updates, { upsert: true })

//Assuming record already existed

> db.iot.findOne()
{
     _id : ObjectId("..."),
     sensor : 5,
     readings : [
          {ts : ISODate("2019-05-10T00:00:00Z"), v : 16},
          {ts : ISODate("2019-05-11T00:00:00Z"), v : 22}
                ],
     count : 2
}
```


Here we start with a reading take from a sensor. Our reading has a value and time it was taken.

We look for a document for that sensor where it currently contains fewer than 200 readings.
If we find one we add the reading to the document and increment the count.
If we don't find one we use upsert to create one.
We assume that we will be retrieving readings over a time period rather than individual readings most of the time.
So we need to index individual readings. In the next pattern we will see how to augment this with metadata.

Computed Pattern

"Never Recompute what you can Precompute"

- Reads are often more common than writes
- Compute on write is less work than compute on read
- Equivalent to OLAP cubes in RDBMS
- When updating the DB – update some summary collections too
- This is one of the caching patterns

- Computed Pattern - prevents having to recompute on query
- Instead, compute on update or on a schedule

Computed Pattern Example

Example: **Exercise Tracking app**

Record user efforts in one collection

Keep other collection to store best times

If a new time recorded by the user is better than the stored record:

$push new time in, $sort and $slice

Saves retrieving best N times at runtime

```
> effort = { athlete:"john",time:920,section: "LDN bridge to eye"}

> db.efforts.insertOne(effort)

> db.best_times.updateOne({ _id: effort.section},
   {$push: {times : {$each:[{ athlete: effort.athlete,
     time : effort.time}], $slice:20, $sort: { "time": 1} }}},{upsert
:true})

> effort = { athlete:"carol",time:840,section: "LDN bridge to eye"}
> db.efforts.insertOne(effort)

> db.best_times.updateOne({ _id: effort.section },
   {$push: {times : { $each:[{athlete: effort.athlete,
     time : effort.time}], $slice:20, $sort: { "time": 1} }}},{upsert
:true})

> db.best_times.findOne()
{ "_id" : "LDN bridge to eye",
"times" : [
              {"athlete" : "carol","time" : 840}
              {"athlete" : "john","time" : 920}]
}
```


With a Computed pattern you keep a separate bit of info computed at write time in the database to avoid computing at read time.

Here we add a runner/riders recorded time to the **efforts** collection in our exercise tracking app.
We update the record in **best_times** for the section they ran/rode if their time is better than any recorded in there.
We add it to the list, sort the list and trim the list to 20 entries with $each and $slice
If the update fails because there is no best_times record for that section yet - it will add one due to upsert

It is possible to optimise this further by checking to see if either their time is faster than one of the times in the record or the record has less than 20 entries.
If you do this then mongodb does not need to perform the editing part as the find fails, if it fails with the upsert clause then it will try to add a record
But that will then fail with a duplicate value of _id - this is an example of having to think about whether errors really are errors.

Computed fields - bucket

Add data summarising the bucket

No additional write costs

Huge savings in read time computation

Add totals (and mean), Max, Min, and even distribution of value histograms for free

```
> db.iot.drop()
> reading={sensor:5,value:22,time:new Date('2019-05-11')}
> query = {sensor:reading.sensor,count: {$lt:200} }

> updates = { $push: {
    readings: { ts: reading.time, v: reading.value }},
    $inc: { count: 1, total: reading.value },
    $max: { max: reading.value }
}

> db.iot.updateOne(query,updates, { upsert: true })
> db.iot.findOne()
{
  _id: ObjectId("..."),
  sensor : 5,
  readings : [{
        v: 16,
        ts: ISODate("2019-05-10T00:00:00Z")
      }, {
        v: 22,
        ts: ISODate("2019-05-11T00:00:00Z")
      }],
        count : 2,
        total : 38,
        max : 22
}
```


This time when we add our reading, we also update a total and max field to allow us to quickly compute a mean for this or for several buckets.
Or the highest value in this one or several buckets.
We could keep the min and max of the timestamp too and index that to allow us to quickly find all values in a time range.
You could also have an array where you increment array[reading.value] by 1 so the array is a histogram of the frequency of values.

Visualise the buckets using this loop. How many documents will it create?
for (let k = 0; k < 256; k++) { reading = { sensor: 5, value: Math.ceil(Math.random() * 30), time: new Date() }; query = { sensor: reading.sensor, count: { $lt: 200 } }; updates = { $push: { readings: { ts: reading.time, v: reading.value } }, $inc: { count: 1 } }; db.iot.updateOne(query,updates, { upsert: true }) }

Versioning Pattern

A good example of Payload versus Processing

- Multiple versions of each document
- At any time, there is one 'current' version
- Can have 'future versions' not yet in current use

Payload – the document you are keeping versions of
Processing – the fields you use to identify the current one
Uses storage – as you store all versions completely, not just deltas

The versioning pattern is a good example of payload vs. processing

Document Version Pattern 1

A simple `latest_version` field is simple but very limited

Handles publishing future versions

Finding 'all' current needs an aggregation with $sort and $first

Indexing is tricky if using valid_to as well

```
> var r = {
  documentid: "secret plan",
  version: 1.0,
  payload: { d: "Some Data" },
  valid_from: new Date('2008-12-14'),
  valid_to: new Date('2099-12-31')
}

> db.versions.drop()
> db.versions.insertOne(r)

//Find the current one at a given time take the highest
//valid_from less than the date wanted. Add a valid_to
// if you want
> now = new Date()
> current = db.versions.find({
  documentid: "secret plan",
  valid_from: { $lt: now }, valid_to: { $gt: now }
}).sort({ valid_from: -1 }).limit(1).pretty()

//Create new version changing valid_from to current date
> r.version=2.0
> r.valid_from=now
> db.versions.insertOne(r)
```


Document version pattern 1 - This is good but if you want to find the latest version of more than a single document at at time you need to use $sort, $group and $first in an aggregation.

It would seem like a good idea to add a field latest: true to each document but this has a couple of limitations. First you need to remove the entry from the old record as well as add it to the new one when adding a new "latest" version - this is not a big issue.

More importantly - you want to be able to add a document now that will automatically become the latest version at some future date, you could do that with a scheduled process or trigger that "moves" the latest flag but better to simply make use of a valid from/valid to date combo.

A Query that looks at two ranges to find if a date is inside them {valid_from: { $lt:x}, valid_to: {glt:x}} is difficult to index well as the query must walk the whole index for one of those clauses, generally put the valid_to first as there will be fewer index entries in the future than the past. it will simply need to check all versions that are not expired already to find the first that's valid already.

Document Version Pattern 2

Simpler option - two collections, easier to query current

Dealing with **future records is difficult** - need to be copied to current when ready

Avoid backend batch/scheduled ops where possible in designs

```
> new_version = {
  documentid: "secret plan",
  version: 1.0,
  payload: { d: "Some Data" },
  valid_from: new Date('2008-12-14'),
  valid_to: Date('2099-12-31')
}

> db.archive.insertOne(new_version)

> db.current.updateOne({ _id: new_version.documentid },
  { $set: new_version },
  { upsert: true })

//But How do we deal with 'future' records?
```


Document version pattern 2

Subset Pattern

Embed important related data and Link to the rest

Use when embedding all data would create very large documents

- Keep array of important subdocuments in the main document
- Migrate remaining subdocuments to another collection

Allows for fast access of important data:

- Recent orders
- Popular social media posts
- Top-rated reviews

Subset Pattern Example

Invoice in full stored in invoices collection

Customer record has just a list of order dates and values

Also limited to the last 3 invoices

```
> let i = {
        _id: new ObjectId(),
        customer: "john page",
        address: "15 Jackton View",
        date: new Date("2025-09-24"),
        items: [  { i: "shoes", q: 1, p: 15 },
                    { i: "socks", q: 5, p: 10 },
                    { i: "shirts", q: 2, p: 60 },
                    { i: "slacks", q: 1, p: 20 }],
        total: 105, status: "delivered"
}

> db.invoices.insertOne(i) //All the data

//Just a date and total in customer record.
> db.customers.updateOne({ _id: i.customer },
    { $push: {recent_invoices:
        {$each: [{id: i._id, date: i.date, spend: i.total
}],
         $slice: -3} } },
    {upsert: true})
```


Example of subset pattern

Extended Reference Pattern

Hybrid of Embedding and Linking data

Use when you need to store data in separate collections, but want to avoid excessive $lookups.

Embed some referenced data in the main document, and $lookup the remaining data from the second collection only when needed

Trade-off is duplicated information.

Movies and actors are different things, so we'll keep them in separate collections. However, actors act in movies, so how can we model that without needing a join?

If you look up a movie, you will most likely see the main cast (and crew). This data is embedded in the movie's document. We also keep an array of _id's of the *full* cast, so that we can perform a lookup on those rare occasions when someone clicks "Full Credits"

Extended Reference Pattern Example

Note that we have duplicated data, but we don't need to run an expensive $lookup call to find the names of the main cast

If a name changes, we would have to update it in all affected documents

Choose a field whose value is unlikely to change to avoid this

```
> db.movies.findOne()
{ _id: ObjectId("6526e76f1f7a9c2d9caf8601"), name: "City
Slickers", year: 1991, directed_by: "Ron Underwood", starring:
["Billy Crystal", "Jack Palance"], full_cast: [345, 588, …, 379,
453, …]}

> db.actors.find({userId: {$in: [345, 588, 379, 453]}})
[{userId: 345, name: "Billy Crystal"}

{userId: 588, name: "Jack Palance"}

{userId: 379, name: "Yeardley Smith"},

{userId: 453, name: "Jake Gyllenhaal"}]
```

Subset vs Extended Reference

Subset

- Only a subset of the data from a related document is embedded within the main document
- Creates partial redundancy by copying part of the secondary document into the main one

Extended Reference

- Includes a reference (_id or another identifier) to the full related document, plus a small amount of embedded data.
- Provides a balance between performance and data normalization

The key difference between subset and the extended reference pattern is that the Subset Pattern stores only the summarized data from the related document without retaining a reference to the full document.
The Extended Reference Pattern stores both a reference to the full document (e.g., an _id) and a summarized subset of its fields. It provides flexibility when you need both summarized fields for quick reads and the ability to retrieve the full document occasionally.

Use Subset Pattern when:

- You only need a summary of related data and don't anticipate needing the full related document.
- You want to optimize for read performance.

Use Extended Reference Pattern when:

- You frequently need related data summaries but occasionally also need the full document.
- You want to balance fast access to commonly accessed fields with the ability to retrieve the full document.

Outlier Pattern

Use when some documents (<5%) are so large that normal operations on them run inefficiently.

- Outlier documents usually contain huge arrays or embedded documents
- Add an identifying flag to the outlier documents and store the array in a separate collection
- App code tests for the flag and makes an additional query (if needed) for the full array

Social networks, where followers are stored in an array, would count celebrity accounts with millions of followers as outliers. Processing or even just retrieving all those followers would cause performance problems. However, for 99% of the accounts, a solution where the followers list is in an embedded array is optimal for our use case.

We create a solution which handles >95% of our documents optimally, and invoke a specialised solution for the outlier documents.

Exercise - Freelancer Scenario

A developer is building an online service for freelancers. Freelancers need to be able to view their total payouts for the week or month. What design pattern would be ideal in implementing this?

A) Embedding Pattern
B) Bucket Pattern
C) Computed Pattern
D) Attribute Pattern

Common Design Anti-Patterns

Things that can make the database slower:

- Very large 1 dimensional arrays
- Long field names
- Lots of fields at the same level / flat documents
- Unnecessary indexes - especially on arrays
- Storing seldom used data fields with frequently used data fields
- Using $lookup instead of using embed objects
- Case insensitive queries without case insensitive indexes

Using large arrays (typically over 200 items) means access times get long for items in the array, prefer shorter arrays with limits to their growth.
Field names take storage so avoid using very long field names
Many fields at the same level take longer to find and retrieve than a document with more structure
Each index we have that we don't need increases time to insert and delete data. An unnecessary index on an Array may have many entries which exacerbates the problem.
If we have some data in a business object that is seldom needed and other parts that are frequently needed, consider splitting the object.
Conversely data which is needed together would be in the same object not fetched with $lookup - do not make a relational schema.
If we need a case insensitive query - make sure we have a case insensitive index.

Exercise - Social network

Build a Twitter-style social network where:

- Users have followers
- Users make posts
- Users view posts from those they follow

Questions to answer:

- What collections would you have, when and how would you update them?
- What edge cases do you need to deal with, and how?
- How will you track followers and following counts?
- How can you keep it fast and efficient?

Come up with a schema for a basic Twitter clone and the queries needed to update it.
Think about what is run on a reading, a post, a follow, and an unfollow

Exercise

Quiz Time!

#1. Which of the following is true about MongoDB Schema Design

A	Schema design revision is important in bad performing applications	B	Schemas should be designed to support how the data is used	C	Schema design should take into account the datatype of values and how frequently the data is accessed
D	Schema design practice recommends one specific schema for any given patterns	E	Schema design best practice recommends to always embed data as nested objects		

Answer in the next slide.

#1. Which of the following is true about MongoDB Schema Design

A. Schema design revision is important in bad performing applications

B. Schemas should be designed to support how the data is used

C. Schema design should take into account the datatype of values and how frequently the data is accessed

D. Schema design practice recommends one specific schema for any given patterns

E. Schema design best practice recommends to always embed data as nested objects

Schema design is very important since it will account for the performance of your application when interacting with the database. That said - if its performing poorly, then it would make sense to revise it (A). Take into factors like how the data is also used (B) and how frequently the data is accessed (C) as best practices. We dont always need to nest - only when it makes sense to and mongodb is flexible so there isn't one schema thats recommended for a given pattern.

#2. What are the benefits of the Bucket pattern versus one document per measurement?

A	Reduced RAM requirements	B	Faster Data Retrieval and summarisation	C	Fixed document structure
D	Reduced contention	E	Making writing data simpler		

Answer in the next slide.

#2. What are the benefits of the Bucket pattern versus one document per measurement?

A Reduced RAM requirements

B Faster Data Retrieval and summarisation

C Fixed document structure

D Reduced contention

E Making writing data simpler

Having both of these design patterns will allow a user to have less RAM requirements since data is already computed beforehand and simply saved on the document. It also means we dont need to run additional logic to summarize the data we want so it will result in faster retrieval and since we are only working on a single document at a time in a given collection as opposed to 2 or more - it greatly reduces the contention.

SKILL BADGE ASSESSMENT – 10 MINUTES

Data Modeling - Relational to Document Model

MongoDB Skill

Data Modeling: Relational to Document Model

Take the Assessment

https://mdb.link/Skill-DMO-RelationalToDocument-PS

Questions

Ask your neighbor

Raise your hand

Let's do it together!

SKILL BADGE ASSESSMENT – 10 MINUTES

Data Modeling - Design Patterns & Anti Patterns

MongoDB Skill

Data Modeling:
Schema Design Patterns
& Anti Patterns

Take the Assessment

https://mdb.link/Skill-DMO-SchemaPatterns-PS

Questions

Ask your neighbor

Raise your hand

Let's do it together!

Recap

Container Types in a collection define embedded (documents) or linked (arrays) models

Schema design focus on the application needs (Payload vs Process fields)

Design patterns can help on the schema design based on use cases

Different Schema Design patterns are:

Attribute pattern, Bucket pattern, Computed pattern, Versioning pattern, Subset pattern, Outlier or Overspill pattern

Answers

Answer - Questions: BSON

How to find a value in a BSON representation?
Start at the beginning, check the field name and either it's right, and that is the field, or jump over that field (using the length if needs be) and read the next field name. Keep checking a list of fields until the field is found.

How long does that take relative to the size of it ?
It's proportional to the number of fields you are looking through multiplied by the number you are looking for (some optimizations apply)

Why have embedded Objects?
An object is a single field with a length - so you can jump the whole object - for example if not looking for address (or a subfield of it) you can skip all address fields.

Answer - Exercise Blog Post Scenario

A developer is designing a blogging platform where users can create posts, comment on them, and like them. The application needs to display each post along with its comments and likes on the same page. What approach should be taken when designing your MongoDB schema to efficiently handle the retrieval of posts with their comments and likes? Assume the application doesn't have many users.

A) Create a separate collection for posts, comments, and likes, and use $lookup during query.

B) Embed comments and likes within the post documents as an array of subdocuments.

C) Store all data in a single large document per user to minimize collections.

D) Use a relational database model to handle the relationships.

Answer - Exercise Freelancer Scenario

A developer is building an online service for freelancers. Freelancers need to be able to view their total payouts for the week or month. What design pattern would be ideal in implementing this?

A) Embedding Pattern
B) Bucket Pattern
C) Computed Pattern
D) Attribute Pattern

The Computed Pattern involves storing pre-calculated totals or aggregates in the database alongside raw data. By computing and storing weekly or monthly payout totals in advance, the system can provide fast and efficient access to these summaries without having to aggregate the results on-the-fly every time a freelancer checks their payouts.

Answer - Exercise: Social Network

db.users

```
{
     _id: String (userid),
     schemaversion: (integer),
     dateCreated: Date(),
     follows: ArrayOfString
     (people followed) [Indexed]
}
```

social.posts

```
{
      _id: ObjectId(),
      schemaVersion: Int,
      postedBy: String,
      date: Date,
      text: String
}
```


We are modeling followers as who someone follows - not who follows them. The very high cardinality model, and one we need to support is millions follow one - and where this happens following and unfollowing should not make changes to one hot record. Also we don't want arrays of millions. You can't limit how many followers someone has.

On the other hand limiting someone to following no more than a couple of thousand people is a reasonable thing to do and if someone is responsible for data about who they follow and it's deleted with them that seems more in line with GDPR etc. If someone deletes themselves and you follow them we can ignore removing them from your followers list, at least asynchronously. Deletion of a user is less common than changing follow/unfollow.